



MARDAS MD2PDF
MARKDOWN TO PDF ENGINE

راهنمای کامل

راهنمای Mardas MD2PDF

راهنمای کامل استفاده و مرجع امکانات

این فایل راهنمای کامل نصب، پیکربندی و استفاده از Mardas MD2PDF است. همین سند به عنوان نمونه زنده رندر نیز استفاده می‌شود و جلد، فهرست مطالب، متن ترکیبی فارسی/English، فرمول، کد، نمودار Mermaid، تصویر، جدول، پانویس، شکست صفحه و HTML امن را نمایش می‌دهد.

مؤسسه

Mardas Lab

تاریخ

۱۴۰۵-۰۲-۳۰

نویسندگان

Mardas MD2PDF تیم
(مستندسازی)

Meraj Rastegar
(mragestsars@gmail.com)

وضعیت

Stable

نسخه

1.13.31

درس / دوره

انتشار حرفه‌ای Markdown

کلیدواژه‌ها

Markdown

PDF

فارسی/English

RTL/LTR

MathJax

فهرست مطالب

۱ معرفی

۱-۱ چک لیست نمونه های رندر

۱-۲ قابلیت های اصلی

۲ نصب

۲-۱ پیش نیازها

۲-۲ نصب از سورس

۲-۳ نصب برای توسعه

۳ اولین خروجی PDF

۴ روند پیشنهادی کار

۵ درباره Front Matter

۵-۱ فیلدهای رایج

۵-۲ رفتار جلد

۶ زبان و جهت سند

۶-۱ اولویت تعیین جهت

۶-۲ نمونه متن ترکیبی

۶-۳ نمونه smoke تصویری فارسی/RTL

۷ فهرست مطالب

۸ مرجع امکانات Markdown

۸-۱ پاراگراف و تاکید

۸-۲ لیست ها

۸-۳ فهرست وظایف

۸-۴ جدول

۸-۵ نقل قول

۸-۶ انواع Callout

۹ فرمول های MathJax

۹-۱ رفع اشکال فرمول ها

۱۰ بلوک های کد

۱۰-۱ بلوک های کد پیشرفته

۱۱ نمودارهای Mermaid

۱۲ تصویر و HTML امن

۱۲-۱ قواعد استفاده از تصویر

۱۲-۲ کد HTML امن

۱۳ امنیت و ورودی‌های قابل اعتماد

۱۳-۱ ناوبری PDF و metadata

۱۴ صفحه‌بندی و Layout

۱۴-۱ شکست صفحه دستی

۱۴-۲ اندازه صفحه

۱۴-۳ انواع Margin

۱۵ اعمال Watermark

۱۶ برزینگ جلد

۱۷ سیستم Appearance

۱۸ روند کار با GUI

۱۹ مرجع CLI

۲۰ اتوماسیون و CI

۲۱ رفع اشکال

۲۱-۱ پیدا نشدن Chromium

۲۱-۲ متن فارسی ظاهر مناسبی ندارد

۲۱-۳ تصویرها دیده نمی‌شوند

۲۱-۴ فرمول‌ها به شکل TeX خام دیده می‌شوند

۲۱-۵ نیاز به بررسی Layout

۲۲ بررسی Preflight فایل PDF

۲۳ پانویس

۲۴ چک‌لیست نهایی انتشار

نکته

این راهنما مرجع کامل کاربر و مرجع امکانات Mardas MD2PDF است. هر قابلیت پشتیبانی‌شده با Markdown قابل اجرا آموزش داده می‌شود و همان نمونه‌ها در PDF رسمی هم دیده می‌شوند.

برنامه Mardas MD2PDF ابزاری برای تبدیل Markdown به PDF است که مخصوص سندهای فارسی، انگلیسی و ترکیبی طراحی شده است. هدف پروژه این است که نویسندگان بتوانند متن را در قالب ساده Markdown بنویسند، اما خروجی نهایی شبیه یک سند PDF مرتب، حرفه‌ای و قابل انتشار باشد.

این پروژه برای گزارش‌های دانشگاهی، مستندات فنی، جزوه‌های آموزشی، راهنماهای نرم‌افزاری، پیش‌نویس‌های پژوهشی، گزارش پروژه و هر سند Markdown که نیاز به خروجی PDF تمیز دارد مناسب است.

TEXT

Markdown -> Structured HTML -> Chromium PDF

در این پروژه متن‌ها مستقیماً روی canvas فایل PDF رسم نمی‌شوند. ابتدا Markdown به HTML ساختاریافته تبدیل می‌شود، سپس CSS چاپی و تنظیمات appearance اعمال می‌شود، فرمول‌ها با MathJax رندر می‌شوند و در مرحله آخر Chromium خروجی PDF را تولید می‌کند. این روش باعث پشتیبانی بهتر از layout چاپی، متن‌های ترکیبی RTL/LTR، فرمول‌های SVG، کدهای هایلایت‌شده، تصویرهای محلی و جدول‌های پیچیده می‌شود.

نکته

این راهنما هم مستند استفاده است و هم نمونه رندر. نسخه PDF همین فایل در پوشه examples/ قرار دارد تا کاربر بتواند خروجی واقعی هر قابلیت مهم را بررسی کند.

پیشنهاد

مستندات کاربر در همین guide نگه داشته می‌شود و صفحه‌های feature/reference موازی حذف شده‌اند تا متن آموزشی و نمونه‌های PDF از هم جدا و ناسازگار نشوند. فایل‌های release، maintenance، security و changelog فقط برای عملیات پروژه در پوشه docs/ باقی مانده‌اند.

چک لیست نمونه‌های رندر

این راهنما عمده‌اً چند نمونه کوچک اما مهم دارد تا کنار راهنمای برنامه، مثل test case تصویری هم عمل کند. وقتی PDF خروجی را بررسی می‌کنید، این موارد را ببینید:

تصویرها، جدول‌ها، بلوک‌های کد و نمودارهای Mermaid دارای caption حالا به صورت بلوک‌های معنایی چاپی نرمال می‌شوند تا توضیح هر بخش در PDF کنار محتوای مربوط به خودش باقی بماند.

بخش نمونه	چیزی که باید در PDF بررسی شود
جلد و metadata	عنوان، زیرعنوان، نویسنده‌ها، خلاصه، نسخه، وضعیت، کلیدواژه و label‌های وابسته به زبان.
فهرست مطالب و outline	شماره‌گذاری تودرتو، لینک‌های داخلی و bookmark‌های PDF viewer که از heading‌های Markdown ساخته می‌شوند.
متن ترکیبی	متن فارسی/English، inline code و شناسه‌ها در یک پاراگراف خوانا بمانند.
فرمول MathJax	فرمول درون‌خطی با متن هماهنگ باشد و فرمول نمایشی وسط‌چین و خوش‌اندازه دیده شود.
بلوک کد	indented، fenced و code block بدون زبان بدون خراب شدن محتوا رندر شوند.
نمودار Mermaid	بلوک کد از نوع <code>flowchart</code> به جای نمایش کد خام، به نمودار SVG تبدیل شود.
تصویر و HTML	تصویر Markdown و تگ امن HTML در PDF دیده شوند.
پانویس و صفحه‌بندی	ارجاع عددی پانویس، لینک بازگشت برای ارجاع‌های تکراری، پانویس چندخطی، شکست صفحه دستی، margin‌ها و شماره صفحه پایدار باشند.
audit فارسی/RTL	نشانه‌گذاری فارسی، عددهای فارسی/لاتین، جدول‌های RTL، عنوان‌های mixed-script در فهرست، back-link و caption پانویس خوانا بمانند.

قابلیت‌های اصلی

قابلیت	توضیح
سند‌های فارسی و انگلیسی	پشتیبانی از <code>lang: fa</code> ، <code>lang: en</code> ، جهت RTL/LTR و متن ترکیبی.
جلد حرفه‌ای	عنوان، زیرعنوان، نویسنده‌ها، خلاصه، لوگو، تاریخ، نسخه، وضعیت، کلیدواژه و metadata آموزشی.
فهرست مطالب و outline	ساخت فهرست چندسطحی و bookmark‌های PDF viewer از heading‌های Markdown.
فرمول MathJax	رندر فرمول‌های درون‌خطی و نمایشی.

قابلیت	توضیح
بلوک کد	هایلایت کدهای fenced و indented با Pygments.
نمودار Mermaid	رندر آفلاین نمودارهای کاربردی <code>graph</code> / <code>flowchart</code> به صورت SVG.
تصویر محلی	جاسازی تصویرهای Markdown و HTML امن به صورت data URI.
کد HTML امن	پاکسازی HTML خام به صورت پیش فرض.
پانویس	پشتیبانی از پانویس‌های چندخطی با محتوای Markdown.
سیستم Appearance	انتخاب جداگانه <code>style</code> ، <code>palette</code> و <code>mode</code> برای شکل سند، رنگ‌بندی و خروجی روشن/تاریک.
اتوماسیون	رابط CLI مناسب برای اسکریپت‌ها و CI.
رابط گرافیکی GUI	رابط گرافیکی محلی برای ویرایش، پیش‌نمایش تقریبی، تنظیم گزینه‌ها و خروجی گرفتن.

نصب

پیش‌نیازها

برای استفاده از پروژه بهتر است این موارد آماده باشند:

- نصب بودن Python نسخه 3.10 یا جدیدتر؛
- محیط مجازی Python؛
- نصب بودن Chromium مربوط به Playwright؛
- فونت مناسب فارسی، ترجیحاً Vazirmatn؛
- نصب بودن Git برای clone کردن پروژه.

■ نصب از سورس

BASH

```
git clone https://github.com/mragetsars/Mardas-MD2PDF.git
cd Mardas-MD2PDF
python -m venv .venv
source .venv/bin/activate
pip install -e .
python -m playwright install chromium
```

در: Windows PowerShell:

POWERSHELL

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -e .
python -m playwright install chromium
```

■ نصب برای توسعه

برای توسعه و اجرای تست‌ها:

BASH

```
pip install -e .[dev]
pytest
```

بعد از نصب، دو دستور اصلی در دسترس است:

دستور	کاربرد
<code>mrs-md2pdf</code>	تبدیل Markdown به PDF از خط فرمان.
<code>mrs-md2pdf-gui</code>	اجرای رابط گرافیکی محلی.

اولین خروجی PDF

تبدیل ساده:

BASH

```
mrs-md2pdf input.md -o output.pdf
```

تبدیل همراه با فهرست مطالب و ظاهر GitHub-style:

BASH

```
mrs-md2pdf input.md -o output.pdf --toc --style modern --palette emerald  
--mode light
```

تولید خروجی طولانی با رفتار شبیه کتاب:

BASH

```
mrs-md2pdf input.md -o output.pdf \  
--toc \  
--toc-depth 4 \  
--toc-page-break \  
--h1-page-break \  
--style textbook \  
--palette slate \  
--mode light
```

ذخیره HTML میانی برای بررسی و رفع اشکال:

BASH

```
mrs-md2pdf input.md -o output.pdf --debug-html output.html
```

نمایش صریح progress bar در ترمینال:

```
mrs-md2pdf input.md -o output.pdf --progress on
```

حالت پیش‌فرض `--progress auto` فقط وقتی progress bar را نشان می‌دهد که دستور در ترمینال تعاملی اجرا شود. برای اسکرپت‌های ساکت‌تر می‌توانید از `--progress off` استفاده کنید.

روند پیشنهادی کار

برای رسیدن به خروجی تمیز، این روند پیشنهاد می‌شود:

قواعد صفحه‌بندی PDF حالا تلاش می‌کنند عنوان‌ها را کنار اولین بلوک بعدی نگه دارند، از تک‌افتادن خط‌های پاراگراف جلوگیری کنند و در صورت طولانی بودن بلوک کد یا جدول، آن‌ها را تمیزتر به صفحه بعد ادامه دهند تا فضای سفید بزرگ ایجاد نشود.

1. محتوای سند را در Markdown بنویسید.

2. در ابتدای فایل، front matter شامل عنوان، نویسنده، زبان، جهت و metadata جلد را اضافه کنید.

3. یک خروجی اولیه با `--toc` و appearance مناسب بگیرید.

4. جلد، فهرست مطالب، صفحه‌های دارای فرمول، صفحه‌های دارای کد، تصویرها و شماره صفحه را بررسی کنید.

5. اگر layout نیاز به بررسی داشت، خروجی `--debug-html` بگیرید و HTML تولیدشده را در مرورگر ببینید.

6. اصلاحات نهایی را در Markdown انجام دهید و PDF را دوباره بسازید.

برای سندهای مهم، بررسی انسانی خروجی نهایی ضروری است. تست خودکار بسیاری از خطاها را می‌گیرد، اما تایپوگرافی، شکست صفحه و فاصله‌ها باید دیده شوند.

درباره Front Matter

بخش Front matter یک بخش YAML اختیاری در ابتدای فایل Markdown است. این بخش جلد، metadata فایل PDF، زبان سند، جهت سند و چند فیلد مخصوص گزارش‌های رسمی یا آموزشی را کنترل می‌کند.

```
---
title: "گزارش فنی من"
subtitle: "Markdown ساخته شده با PDF یک سند"
authors:
  - name: "Mardas"
    email: "mragetsars@gmail.com"
    affiliation: "Mardas Lab"
    role: "Author"
summary: |
  این متن روی جلد نمایش داده می شود.
  متن چند خطی پشتیبانی می شود
institution: "نام دانشگاه یا سازمان"
department: "نام دانشکده یا دپارتمان"
course: "نام درس یا پروژه"
supervisor: "نام استاد یا راهنما"
date: "۱۴۰۵-۰۲-۳۰"
version: "1.13.31"
status: "Draft"
keywords: [Markdown, PDF, RTL, MathJax]
cover_label: "گزارش فنی"
branding:
  mode: full
brand:
  name: "آزمایشگاه پژوهشی Acme"
  logo: "assets/acme-logo.svg"
  footer: "گزارش فنی داخلی"
lang: fa
dir: rtl
---
```

فیلدهای رایج

کاربرد	فیلد
عنوان جلد و title در metadata فایل PDF.	title
متن اختیاری زیر عنوان اصلی.	subtitle
نویسنده ساده و تکمقداری.	author
فهرست نویسنده ها با <code>name</code> ، <code>email</code> ، <code>affiliation</code> و <code>role</code> .	authors

کاربرد	فیلد
خلاصه روی جلد و subject در metadata.	<code>summary</code> / <code>description</code>
اطلاعات دانشگاهی یا سازمانی.	<code>institution</code> , <code>department</code> , <code>course</code>
فیلدهای اختیاری برای گزارش‌های آموزشی.	<code>supervisor</code> , <code>group</code> , <code>student_id</code>
کارت‌های وضعیت سند روی جلد.	<code>date</code> , <code>version</code> , <code>status</code>
کلیدواژه‌های جلد و metadata فایل PDF.	<code>keywords</code> / <code>tags</code>
برچسب کوچک بالای عنوان جلد.	<code>cover_label</code>
حالت برندینگ جلد: <code>off</code> , <code>subtle</code> یا <code>full</code> . مقدار پیش‌فرض <code>off</code> است.	<code>branding.mode</code>
برند سازمانی اختیاری که در صورت فعال بودن branding روی جلد می‌آید.	<code>brand.name</code> , <code>brand.logo</code> , <code>brand.footer</code>
مسیر لوگوی سفارشی قدیمی/ساده نسبت به فایل Markdown. برای سندهای جدید <code>brand.logo</code> تمیزتر است.	<code>cover_logo</code> / <code>logo</code>
زبان داخلی رابط سند، معمولاً <code>fa</code> یا <code>en</code> .	<code>lang</code>
جهت پوسته سند: <code>auto</code> , <code>ltr</code> یا <code>rtl</code> .	<code>dir</code>

■ رفتار جلد

جلد جدا از محتوای اصلی رندر می‌شود. نتیجه این تصمیم چنین است:

- جلد جزو شماره صفحات محتوایی حساب نمی‌شود؛
- شماره‌گذاری footer از صفحه بعد از جلد شروع می‌شود؛
- طرح watermark فقط روی صفحات محتوایی اعمال می‌شود؛
- style خروجی می‌تواند برای جلد پس‌زمینه تمام‌صفحه داشته باشد؛
- در صورت نیاز، جلد کاملاً قابل حذف است.

حذف جلد:

BASH

```
mrs-md2pdf input.md -o output.pdf --no-cover
```

جلد به صورت پیش‌فرض بدون برندینگ خروجی می‌گیرد تا سند عادی شبیه تبلیغ ابزار نباشد. برندینگ کامل را فقط وقتی فعال کنید که واقعاً لازم است:

BASH

```
mrs-md2pdf input.md -o output.pdf --branding full --brand-name "Acme Research Lab"
```

استفاده از لوگوی برند سفارشی:

BASH

```
mrs-md2pdf input.md -o output.pdf --branding full --brand-logo ./assets/logo.png
```

حفظ جلد بدون نمایش لوگو:

BASH

```
mrs-md2pdf input.md -o output.pdf --no-cover-logo
```

برای یادداشت کوچک تولیدشده با ابزار از `branding.mode: subtle` استفاده کنید. راهنماهای رسمی پروژه چون خود Mardas MD2PDF را مستند می‌کنند، به صورت صریح `branding.mode: full` دارند.

زبان و جهت سند

کنترل جهت یکی از مهم‌ترین بخش‌های تولید PDF فارسی/انگلیسی است. در Mardas MD2PDF زبان سند و جهت سند از هم جدا در نظر گرفته شده‌اند.

باعث ایجاد پوسته فارسی/RTL، لیبل‌های فارسی جلد، عنوان فارسی callout ها و عنوان `lang: fa` می‌شود. **فهرست مطالب**

باعث ایجاد پوسته انگلیسی/LTR، لیبل‌های انگلیسی جلد، عنوان انگلیسی callout ها و عنوان `lang: en` می‌شود. **Table of Contents**

اولویت تعیین جهت

جهت سند با این ترتیب مشخص می‌شود:

1. گزینه خط فرمان `--dir rtl|ltr|auto`

2. فیلدهای front matter مثل `dir`، `direction`، `text_direction` یا `document_direction`

3. پیش‌فرض به‌دست‌آمده از `lang`

4. تشخیص خودکار از متن Markdown

■ نمونه متن ترکیبی

در متن انگلیسی می‌توان واژه‌های فارسی مثل راست به چپ، فونت فارسی و گزارش فنی را آورد بدون اینکه جمله انگلیسی به هم بریزد.

در متن فارسی نیز می‌توان شناسه‌های English مثل `Playwright`، `MathJax`، `GitHub Actions`، `PDF` و `RTL/LTR` را داخل همان پاراگراف استفاده کرد.

همچنین Inline code هم پایدار می‌ماند: `mrs-md2pdf input.md -o output.pdf --toc`.

■ نمونه smoke تصویری فارسی/RTL

این نمونه کوچک عمداً داخل guide مانده است، چون guide هم راهنمای کاربر است و هم test case زنده ^۱ renderer. این بخش نشانه‌گذاری فارسی، نام‌های لاتین، عدد فارسی، caption جدول، و سلول‌های mixed-direction را در PDF رسمی نگه می‌دارد.

۱. این نمونه زنده عمداً نشانه‌گذاری فارسی، شناسه‌های لاتین، عددهای فارسی، caption جدول و پانویس‌های محلی همان صفحه را داخل راهنمای رسمی فارسی نگه می‌دارد.

آیا خروجی PDF برای `version 1.13.31` و شماره ۱۴۰۵ پایدار است؟ پاسخ: بله؛ جدول زیر باید hook‌های RTL، mixed-script و mixed-number را فعال کند.

جدول ۱۲. نمونه جدول فارسی/RTL با عددهای ترکیبی.			
بخش نمونه	مقدار	انتظار در PDF	
شماره فارسی	۱۴۰۵	عدد فارسی کنار متن RTL پایدار بماند.	
نسخه فنی	version 1.13.31 و ۱.۹.۹	عددهای Latin/Persian در یک سلول خوانا بمانند.	
شناسه انگلیسی	<code>MathJax</code> ، <code>TOC</code> ، <code>PDF</code>	identifierهای English داخل جدول فارسی جابه‌جا نشوند.	

همین بخش چک‌لیست فارسی/RTL برای شناسه‌های ترکیبی، سبک عددها، جدول‌های RTL، captionها و لینک‌های بازگشت پانویس است. راهنمای انگلیسی هم یک نمونه smoke کوچک از همین رفتارها دارد.

فهرست مطالب

فعال کردن فهرست مطالب:

BASH

```
mrs-md2pdf input.md -o output.pdf --toc
```

تعیین عمق فهرست:

BASH

```
mrs-md2pdf input.md -o output.pdf --toc --toc-depth 3
```

شروع متن اصلی از صفحه جدید بعد از فهرست:

BASH

```
mrs-md2pdf input.md -o output.pdf --toc --toc-page-break
```

شروع هر heading سطح اول از صفحه جدید:

BASH

```
mrs-md2pdf input.md -o output.pdf --h1-page-break
```

فهرست مطالب از heading های Markdown ساخته می شود و فرمول های درون خطی داخل heading ها مثل $E = mc^2$ و ϵ را خوانا نگه می دارد.

لینک های فهرست مطالب چاپی و bookmark های PDF viewer از مقصدهای ثابت همان heading ها استفاده می کنند؛ بنابراین کلیک روی هر لایه ناوبری کاربر را به عنوان واقعی در متن سند می برد، نه به ردیف مشابه داخل خود فهرست مطالب.

مرجع امکانات Markdown

پاراگراف و تاکید

پاراگراف‌های Markdown، **متن bold**، متن *italic*، `inline code`، لینک، لیست مرتب، لیست نامرتب، task list و نقل قول پشتیبانی می‌شوند.

لیست‌ها

- نوشتن محتوای اصلی با Markdown.
- افزودن front matter برای metadata.
- انتخاب appearance مناسب.
- بررسی خروجی نهایی PDF.

1. نصب پکیج.

2. نصب Chromium.

3. اجرای CLI.

4. بررسی خروجی.

فهرست وظایف

- ورودی Markdown ☒
- تایپوگرافی فارسی/English ☒
- رندر MathJax ☒
- هایلایت کد ☒
- بازبینی انسانی نهایی ☐

جدول

قابلیت	وضعیت	توضیح
متن ترکیبی RTL/LTR	بله	پاراگراف، heading، لیست و سلول جدول با direction-aware styling رندر می‌شوند.
تصویر محلی	بله	تصویرهای Markdown و HTML امن در صورت امکان embed می‌شوند.
فرمول MathJax	بله	فرمول درون‌خطی و نمایشی اندازه‌گذاری جدا دارند.
هایلایت کد	بله	برای fenced code block و indented code block از Pygments استفاده می‌شود.
نمودار Mermaid	بله	fence های کاربردی <code>flowchart</code> و <code>graph</code> به نمودار SVG تبدیل می‌شوند.
پانویس	بله	پانویس چندخطی با Markdown داخلی پشتیبانی می‌شود.
کد HTML خام امن	بله	tag ها و event handler های خطرناک حذف می‌شوند.

نقل قول

کیفیت خروجی PDF فقط تبدیل متن نیست. تایپوگرافی، line height، کنتراست، صفحه‌بندی، تصویرها، فرمول‌ها و قابل پیش‌بینی بودن رندر اهمیت دارند.

انواع Callout

انواع Callout ها از marker های مشابه GitHub استفاده می‌کنند و عنوان آن‌ها با توجه به زبان سند ترجمه می‌شود.

نکته

برای نکته‌های توضیحی که باید از متن اصلی جدا دیده شوند، از callout استفاده کنید.

پیشنهاد

وقتی لازم است HTML دقیق ارسال شده به Chromium را ببینید، از `--debug-html output.html` استفاده کنید.

گزینه --unsafe-html فقط برای فایل‌های محلی قابل اعتماد مناسب است، چون sanitizer داخلی را غیرفعال می‌کند.

فرمول‌های MathJax

فرمول‌های درون‌خطی باید با ارتفاع خط اطراف هماهنگ باشند: $E = mc^2$ ، $T = 500$ و $\Sigma = I \cdot \epsilon$ باید طبیعی وسط جمله قرار بگیرند.

فرمول نمایشی فضای بیشتری می‌گیرد:

$$\int_{-\infty}^{\infty} e^{-x^2} dx = \sqrt{\pi}$$

نمونه ماتریسی:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad \det(A) = -2$$

نمونه: aligned

$$\text{precision} = \frac{TP}{TP + FP}$$

$$\text{recall} = \frac{TP}{TP + FN}$$

رفع اشکال فرمول‌ها

اگر فرمول‌ها به شکل TeX خام دیده شدند:

- مطمئن شوید `--no-mathjax` استفاده نشده باشد؛
- هشدارهای ترمینال را بررسی کنید؛
- برای سندهای خیلی بزرگ مقدار `timeout` مرورگر را افزایش دهید؛
- یک فایل کوچکتر بسازید تا فرمول مشکل‌دار را جدا کنید.

بلوک‌های کد

بلوک‌های کد fenced از برجسته‌سازی نحوی و برچسب زبان پشتیبانی می‌کنند.

PYTHON

```
from dataclasses import dataclass

@dataclass
class Document:
    title: str
    lang: str = "fa"

def render_message(doc: Document) -> str:
    return f"Rendering {doc.title} as PDF"

print(render_message(Document("راهنمای Mardas")))
```

JAVASCRIPT

```
const items = ["Markdown", "Persian", "English", "MathJax", "PDF"];
const message = items.map((item, index) => `${index + 1}.
${item}`).join("\n");
console.log(message);
```

بلوک کد fenced بدون زبان هم معتبر است:

TEXT

این بلوک زبان مشخصی ندارد.
با این حال باید بدون خطا رندر شود.

همچنین indented code block نیز پشتیبانی می‌شود:

TEXT

```
SELECT title, lang, version
FROM documents
WHERE renderer = 'mardas-md2pdf';
```

توجه کنید که inline code در برابر پردازش فرمول و پانویس محافظت می‌شود. یعنی `$x$` و `[^note]` وقتی داخل backtick باشند، به شکل literal باقی می‌مانند.

■ بلوک‌های کد پیشرفته

برای آموزش، بازبینی کد یا مستندات فنی، بلوک‌های کد می‌توانند عنوان فایل، شماره خط و خطوط برجسته‌شده داشته باشند.

MARKDOWN

```
```python title="renderer.py" {2,5-6} linenos
def convert(markdown: str) -> bytes:
 html = render_markdown(markdown)
 pdf = render_pdf(html)
 metadata = inspect_pdf(pdf)
 log_export(metadata)
 return pdf
```
```

بخش `{2,5-6}` یعنی: خط ۲ و بازه‌ی شامل خط‌های ۵ تا ۶ برجسته شوند. شماره‌های highlight بر اساس ردیف‌های داخل همان snippet حساب می‌شوند، مگر اینکه برای شماره‌گذاری فایل اصلی از `linenostart` استفاده شده باشد.

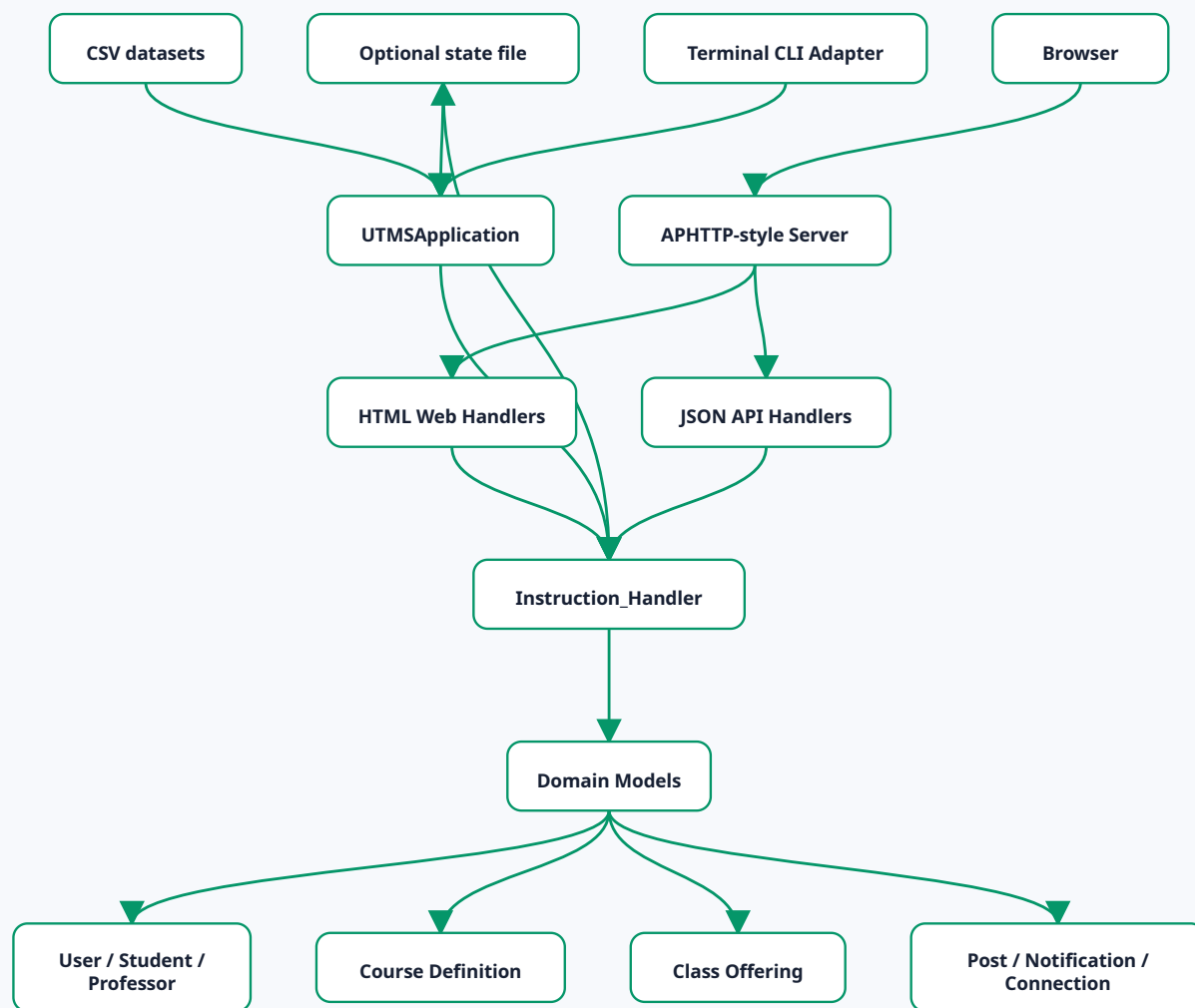
نمونه بالا در PDF به شکل یک بلوک کد با caption سفارشی، شماره خط و highlight رندر می‌شود:

```
1 def convert(markdown: str) -> bytes:
2     html = render_markdown(markdown)
3     pdf = render_pdf(html)
4     metadata = inspect_pdf(pdf)
5     log_export(metadata)
6     return pdf
```

نمودارهای Mermaid

بلوک‌های Mermaid برای flowchart به صورت SVG در HTML میانی رندر می‌شوند و بعد داخل PDF قرار می‌گیرند. تمرکز renderer فعلاً روی زیرمجموعه رایج در مستندات پروژه است: `graph` / `flowchart`، جهت‌های `BT`، `TB`، `TD`، `RL`، `LR`، `node`های مستطیلی، گرد، دایره‌ای و لوزی، `edge`های ساده، `dotted`، ضخیم و `edge`های دارای `label`. نکته مهم در خروجی PDF این است که بلوک زیر باید به شکل نمودار دیده شود، نه کد خام:

MERMAID



این نمونه کوچکتر برای بررسی label روی edge و شکل node ها مفید است:

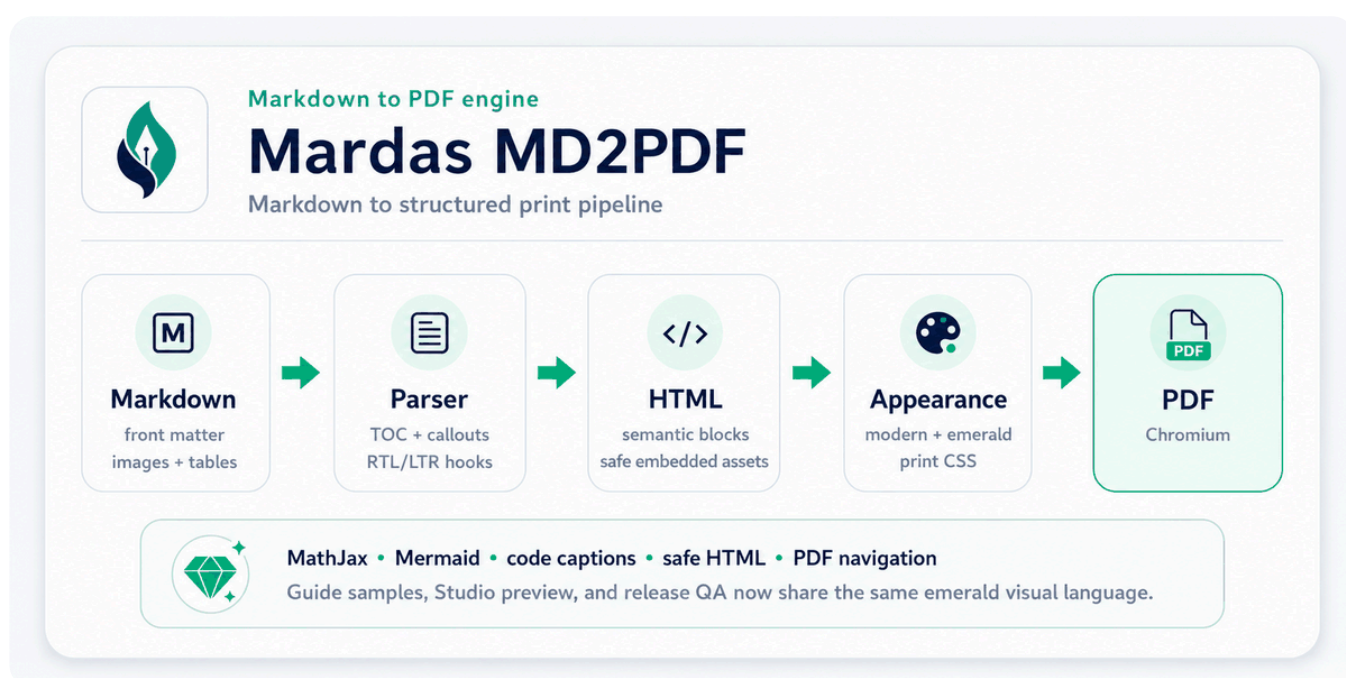
MERMAID



اگر یک بلوک Mermaid از syntax های پیشرفته خارج از این زیرمجموعه استفاده کند، بهتر است تا زمان اضافه شدن آن قابلیت، یک تصویر fallback کوچک از همان نمودار کنار Markdown نگه داشته شود. برای معماری های معمول، data flow و نمودارهای آموزشی، renderer داخلی کافی است و به اینترنت نیاز ندارد.

تصویر و HTML امن

مسیر تصویرها نسبت به فایل Markdown resolve می شود. تصویرهای محلی اگر حجم مناسبی داشته باشند داخل HTML/PDF نهایی embed می شوند.



شکل ۱. نمای کلی معماری.

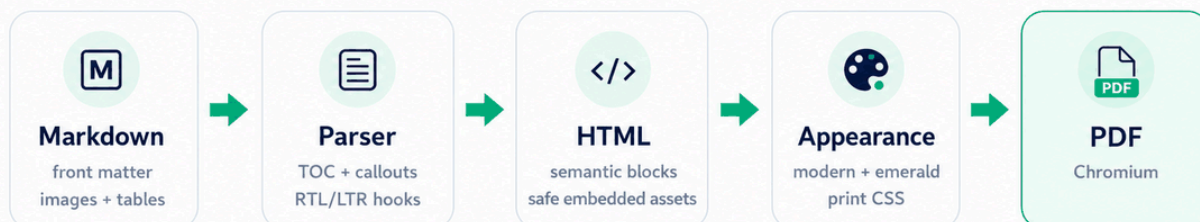
وقتی اندازه دهی دقیق تر لازم است، می توان از HTML امن استفاده کرد. برای اینکه نمونه HTML همان زبان بصری شکل بالا را حفظ کند، در اینجا هم از همان بنر/نمودار اولیه استفاده می شود و دیگر از لوگوی خام استفاده نمی کنیم:



Markdown to PDF engine

Mardas MD2PDF

Markdown to structured print pipeline



MathJax • Mermaid • code captions • safe HTML • PDF navigation

Guide samples, Studio preview, and release QA now share the same emerald visual language.

قواعد استفاده از تصویر

- برای خروجی پایدار PDF، تصویرهای محلی بهتر از تصویرهای remote هستند.
- قبل از embed کردن، حجم تصویرها را منطقی نگه دارید.
- مسیر تصویر را نسبت به محل فایل Markdown بنویسید.
- مسیر تصویر را داخل پوشه همان سند نگه دارید. مسیرهای absolute، URLهای `file:`، خروج از پوشه با الگوهایی مثل `../secret.png` و fallback به current working directory به صورت پیش فرض block می شوند.
- اگر یک تصویر محلی به شکل امن embed نشود، با یک placeholder شفاف جایگزین می شود تا Chromium آن را از طریق `<base>` سند load نکند.
- در GUI، فایل ها یا پوشه تصویر را قبل از export attach کنید.
- تصویرهای خیلی بزرگ block می شوند و renderer برای جلوگیری از مصرف زیاد حافظه هشدار می دهد.

کد HTML امن

کد HTML خام به صورت پیش فرض sanitize می شود. sanitizer عناصر مناسب سند مثل `<div>`، `<span>`، `<table>`، `<figure>` و `<img>` را نگه می دارد، اما محتوای فعال یا خطرناک مثل `script`، `event handler`، `iframe`، `form`، `stylesheet` خارجی، URLهای `file:`، URL scheme نامن و `data:` image های غیر-raster را حذف می کند.

image **data:** امن فقط برای format های raster رایج مثل PNG، JPEG، GIF، WebP، BMP و AVIF مجاز است. data URL از نوع SVG در HTML امن block می‌شود؛ برای نمودارها بهتر است از فایل محلی قابل اعتماد یا بلوک Mermaid استفاده کنید.

غیرفعال کردن sanitizer فقط برای فایل‌های قابل اعتماد:

BASH

```
mrs-md2pdf input.md -o output.pdf --unsafe-html
```

امنیت و ورودی‌های قابل اعتماد

Mardas MD2PDF یک ابزار انتشار محلی است. فایل Markdown، assetهای attach شده در GUI، لوگوی جلد، watermark و HTML خام را محتوای نویسنده و قابل اعتماد در نظر بگیرید؛ مگر اینکه renderer را داخل محیط جداشده اجرا کنید.

رندر پیش‌فرض یک مرز محافظه‌کارانه برای فایل‌ها نگه می‌دارد: تصویرهای محلی نسبت به محل فایل Markdown resolve می‌شوند، اگر امن باشند به `data:` URL تبدیل می‌شوند و اگر به بیرون از پوشه سند اشاره کنند یا قابل embed نباشند block می‌شوند. HTML خام هم به صورت پیش‌فرض sanitize می‌شود، مگر اینکه `--unsafe-html` را فعال کنید.

حالت sandbox کرومیوم با `--chromium-sandbox` کنترل می‌شود:

| حالت | رفتار |
|-------------------|--|
| <code>auto</code> | برای کاربر عادی sandbox را روشن نگه می‌دارد و فقط هنگام اجرای root آن را خاموش می‌کند. |
| <code>on</code> | همیشه sandbox کرومیوم را درخواست می‌کند. |
| <code>off</code> | گزینه <code>--no-sandbox</code> را به Chromium می‌دهد؛ فقط در containerهای قابل اعتماد یا CI جداشده استفاده شود. |

برای سند های نامطمئن، خروجی را داخل container یا محیط disposable بسازید و `--unsafe-html` را خاموش نگه دارید. جزئیات کامل در `./SECURITY.md` آمده است.

■ ناوبری PDF و metadata

PDFهای تولیدشده metadata استاندارد سند را از front matter می‌گیرند و از headingهای Markdown یک outline برای PDF viewer می‌سازند. فهرست مطالب چاپی همچنان داخل متن سند دیده می‌شود، اما outline کنار PDF به خواننده کمک می‌کند سریع بین بخش‌ها جابه‌جا شود.

وقتی فهرست مطالب چاپی هم لازم دارید از `--toc` استفاده کنید. outline از همان مجموعه heading ساخته می‌شود تا فهرست چاپی و bookmarkهای PDF با هم هماهنگ بمانند.

صفحه‌بندی و Layout

■ شکست صفحه دستی

برای شکست صفحه دستی می‌توانید HTML امن زیر را قرار دهید:

HTML

```
<div class="md2pdf-page-break"></div>
```

■ اندازه صفحه

استفاده از اندازه نام‌دار:

BASH

```
mrs-md2pdf input.md -o output.pdf --page-size A4
```

استفاده از حالت: landscape

BASH

```
mrs-md2pdf input.md -o output.pdf --page-size "A4 landscape"
```

استفاده از ابعاد دقیق:

BASH

```
mrs-md2pdf input.md -o output.pdf --page-size "210mm 297mm"
```

BASH

```
mrs-md2pdf input.md -o output.pdf \  
--margin-top 18mm \  
--margin-bottom 18mm \  
--margin-x 16mm
```

اعمال Watermark

اعمال Watermark متنی:

BASH

```
mrs-md2pdf input.md -o output.pdf --watermark "DRAFT"
```

اعمال Watermark تصویری:

BASH

```
mrs-md2pdf input.md -o output.pdf \  
--watermark-image ./src/mardas_md2pdf/assets/mardas-md2pdf-logo.png \  
--watermark-opacity 0.05 \  
--watermark-width 95mm
```

طرح Watermark فقط روی صفحات محتوایی اعمال می‌شود و روی جلد قرار نمی‌گیرد.

برندینگ جلد

خروجی‌های PDF به صورت پیش‌فرض بدون برندینگ ساخته می‌شوند. این رفتار باعث می‌شود گزارش، جزوه یا سند کاری کاربر شبیه تبلیغ محصول نباشد و مالکیت بصری سند برای خود کاربر بماند.

YAML

```
branding:
  mode: off
```

حالت‌های موجود:

حالت	کاربرد
off	پیش‌فرض برای گزارش‌ها و جزوه‌های عادی.
subtle	یادداشت کوچک generated-with برای پیش‌نویس‌های غیررسمی.
full	برندینگ عمدی پروژه یا سازمان.

نمونه برند سازمانی:

YAML

```
branding:
  mode: full
brand:
  name: "Acme Research Lab"
  logo: "assets/acme.png"
  footer: "Internal Technical Report"
```

معادل CLI:

BASH

```
mrs-md2pdf input.md -o output.pdf \
  --branding full \
  --brand-name "Acme Research Lab" \
  --brand-logo assets/acme.png \
  --brand-footer "Internal Technical Report"
```

راهنماهای فارسی و انگلیسی پروژه چون نمونه رسمی Mardas MD2PDF هستند، به شکل صریح از `branding.mode: full` استفاده می‌کنند.

سیستم Appearance

برنامه Mardas MD2PDF به جای چند سیستم موازی، یک مدل ظاهر دارد: **style** شکل و layout سند را کنترل می‌کند، **palette** رنگ‌های accent را تعیین می‌کند و **mode** خروجی روشن یا تاریک را انتخاب می‌کند.

Style	کاربرد پیشنهادی
modern	مستندات عمومی، proposal، گزارش نرم‌افزاری و راهنمای محصول.
github	مستندات پروژه‌ای شبیه README و خروجی نزدیک به GitHub-style Markdown.
textbook	جزوه‌های آموزشی طولانی و محتوای آموزشی فارسی/English.
academic	گزارش رسمی، سند دانشگاهی، پیش‌نویس پایان‌نامه و مقاله ساختاریافته.

Palette	حس رنگی
blue	آبی حرفه‌ای پیش‌فرض.
emerald	سبز آرام برای گزارش‌ها و داشبوردها.
violet	بنفش خلاقانه برای سندهای محصولی.
amber	رنگ گرم برای محتوای آموزشی و review.
rose	رنگ برجسته برای گزارش‌های editorial.
slate	خنثی و رسمی برای مستندات فنی.
neutral	خاکستری مینیمال برای خروجی رسمی.

انتخاب appearance از CLI:

BASH

```
mrs-md2pdf input.md -o output.pdf --style modern --palette emerald --mode light
mrs-md2pdf input.md -o output.pdf --style academic --palette emerald --mode dark
```

یا ذخیره در: front matter

YAML

```
appearance:
  style: modern
  palette: emerald
  mode: light
```

برای دیدن گزینه‌ها از `--list-styles`، `--list-palettes` و `--list-modes` استفاده کنید.

حالت تاریک برای هر style سطح رنگی مخصوص خودش را دارد. `modern` از پس‌زمینه سرمه‌ای عمیق استفاده می‌کند، `github` به سطح تاریک شبیه GitHub نزدیک است، `textbook` ظاهر تقریباً سیاه و هماهنگ با جلد قدیمی را نگه می‌دارد و `academic` پس‌زمینه ذغالی گرم دارد. Palette همچنان رنگ accent را در حالت روشن و تاریک، از جمله تزئینات جلد و callout، کنترل می‌کند.

روند کار با GUI

اجرای GUI:

BASH

```
mrs-md2pdf-gui
```

رابط کاربری GUI برای کاربرانی مناسب است که روند بصری را ترجیح می‌دهند:

1. نوشتن یا paste کردن. Markdown.

2. تکمیل بخش **Document** برای عنوان، نویسنده، نام فایل خروجی، اندازه صفحه و جهت.

3. انتخاب کارتهای **Appearance** برای style، palette و mode روشن/تاریک.

4. استفاده از **Branding** فقط وقتی PDF باید نشان سازمان، محصول یا آزمایشگاه داشته باشد.

5. استفاده از **Layout** برای فهرست مطالب، جلد و جریان صفحه‌ها.

6. باز کردن **Advanced** فقط برای watermark، حذف شماره صفحه یا attach کردن assetهای محلی.

7. استفاده از preview پیش‌فرض PDF-like برای HTML تولیدشده توسط renderer با شبیه‌سازی اندازه صفحه، margin، scale شدن صفحه و markerهای مرز صفحه؛ برای بازخورد بسیار سریع هنگام ویرایش، استفاده از Fast preview.

8. استفاده از **S+Ctrl/Cmd** برای ذخیره Markdown و **Enter+Ctrl/Cmd** برای export سریع PDF.

Studio پیش‌نویس فعلی، layout، حالت روشن/تاریک، جهت preview، عرض editor و گزینه‌های export را در local storage مرورگر نگه می‌دارد. این کار باعث می‌شود refresh ناخواسته صفحه در جلسه‌های ویرایش طولانی کمتر آزاردهنده باشد. برای پاک کردن پیش‌نویس محلی و برگشتن به حالت تمیز از **Reset State** استفاده کنید. همین بخش مرجع روند کار با Studio برای کاربر است.

اگر export با خطا روبه‌رو شود، Studio وضعیت HTTP و کد پایدار backend مثل `invalid_json`، `invalid_page_size`، `invalid_toc_depth`، `invalid_watermark_opacity`، `markdown_too_large` یا `render_failed` را نشان می‌دهد. اگر GUI را روی host غیرلوکال bind کنید، backend هشدار می‌دهد؛ چون کاربران قابل دسترسی در شبکه می‌توانند Markdown و asset بفرستند.

مهم

Studio اکنون به‌صورت پیش‌فرض preview نوع PDF-like با scale خودکار صفحه و markerهای مرز صفحه را نشان می‌دهد. Fast preview هنوز برای ویرایش فوری مفید است، اما PDF نهایی همچنان توسط backend renderer و layout چاپی Chromium ساخته می‌شود.

مرجع CLI

گزینه	توضیح
<code>input</code>	فایل Markdown ورودی.
<code>--output</code> ، <code>-o</code>	مسیر PDF خروجی.
<code>--description</code> ، <code>--author</code> ، <code>--title</code>	override کردن metadata موجود در front matter.
<code>--toc-depth</code> ، <code>--toc</code>	فعال‌سازی و تنظیم فهرست مطالب.
<code>--h1-page-break</code> ، <code>--toc-page-break</code>	کنترل صفحه‌بندی چاپی.

گزینه	توضیح
<code>--style</code>	انتخاب <code>academic</code> یا <code>textbook</code> ، <code>github</code> ، <code>modern</code>
<code>--palette</code>	انتخاب <code>rose</code> ، <code>amber</code> ، <code>violet</code> ، <code>emerald</code> ، <code>blue</code> ، <code>slate</code> یا <code>neutral</code>
<code>--mode</code>	انتخاب <code>light</code> یا <code>dark</code>
<code>--list-modes</code> ، <code>--list-palettes</code> ، <code>--list-styles</code>	نمایش گزینه‌های appearance و خروج.
<code>--page-size</code>	اندازه صفحه مثل <code>A4</code> ، <code>Legal</code> ، <code>Letter</code> ، <code>A4 landscape</code> یا ابعادی مثل <code>210mm 297mm</code>
<code>--dir</code>	اجبار جهت به <code>rtl</code> یا <code>ltr</code> ، <code>auto</code>
<code>--margin-top</code> ، <code>--margin-bottom</code> ، <code>--margin-x</code>	کنترل margin صفحه.
<code>--font-dir</code>	مسیر فونت‌های محلی. Vazirmatn.
<code>--chromium-path</code>	مسیر سفارشی Chromium یا Chrome.
<code>--chromium-sandbox</code>	حالت sandbox مرورگر: <code>auto</code> ، <code>on</code> یا <code>off</code> . مقدار پیش‌فرض: <code>auto</code>
<code>--debug-html</code>	ذخیره HTML میانی.
<code>--no-cover</code> ، <code>--branding</code> ، <code>--brand-name</code> ، <code>--brand-logo</code> ، <code>--brand-footer</code> ، <code>--no-cover-logo</code>	تنظیم جلد و برندینگ صریح.
<code>--watermark</code> ، <code>--watermark-image</code>	افزودن watermark متنی یا تصویری با لایه‌بندی هماهنگ با mode.
<code>--allow-remote-assets</code>	اجازه به سند‌های قابل اعتماد برای بارگذاری تصویرهای remote <code>http(s)</code> . پیش‌فرض غیرفعال است.
<code>--no-header-footer</code>	حذف footer چاپی.
<code>--no-mathjax</code>	غیرفعال کردن MathJax.
<code>--unsafe-html</code>	غیرفعال کردن sanitization برای فایل‌های قابل اعتماد.
<code>--timeout-ms</code>	timeout مرورگر بر حسب میلی‌ثانیه.
<code>--progress</code>	حالت progress bar ترمینال: <code>auto</code> ، <code>on</code> یا <code>off</code> . مقدار پیش‌فرض: <code>auto</code>

برای دیدن همه گزینه‌ها:

BASH

```
mrs-md2pdf --help
```

اتوماسیون و CI

یک دستور ساده برای ساخت PDF در اسکریپت:

BASH

```
mrs-md2pdf docs/report.md -o build/report.pdf --toc --style modern --  
palette emerald --mode light
```

مخزن شامل workflow گیت‌هاب Actions است که Ruff، pytest و یک smoke test واقعی رندر با Chromium را روی نسخه‌های پشتیبانی‌شده Python اجرا می‌کند. برای CI بهتر است گزینه‌ها صریح باشند:

BASH

```
mrs-md2pdf docs/report.md -o build/report.pdf \  
--toc \  
--toc-depth 4 \  
--style modern \  
--palette emerald \  
--mode light \  
--page-size A4 \  
--dir auto \  
--timeout-ms 60000 \  
--progress off
```

برای اشکال‌زدایی در CI، HTML میانی را به عنوان artifact نگه دارید:

BASH

```
mrs-md2pdf docs/report.md -o build/report.pdf --debug-html  
build/report.html
```

رفع اشکال

پیدا نشدن Chromium

این دستور را اجرا کنید:

BASH

```
python -m playwright install chromium
```

یا مسیر مرورگر را صریح بدهید:

BASH

```
mrs-md2pdf input.md -o output.pdf --chromium-path /path/to/chrome
```

■ متن فارسی ظاهر مناسبی ندارد

یک فونت فارسی مناسب مثل Vazirmatn نصب کنید یا مسیر فونت را بدهید:

BASH

```
mrs-md2pdf input.md -o output.pdf --font-dir ./fonts
```

اگر مسیر فونت وجود نداشته باشد یا فایل شناخته شده‌ای داخل آن پیدا نشود، renderer هشدار می‌دهد و از فونت‌های سیستم استفاده می‌کند.

■ تصویرها دیده نمی‌شوند

مطمئن شوید مسیر تصویرها نسبت به فایل Markdown درست است و داخل پوشه همان سند باقی می‌ماند. مسیرهای absolute، URL‌های `file:`، خروج از پوشه با `...`، فایل‌های گم‌شده و تصویرهای خیلی بزرگ با placeholder امن جایگزین می‌شوند تا Chromium آن‌ها را از فایل سیستم load نکند. اگر از GUI استفاده می‌کنید، فایل‌ها یا پوشه‌های تصویر را قبل از خروجی گرفتن attach کنید و هشدارهای renderer را بررسی کنید.

■ فرمول‌ها به شکل TeX خام دیده می‌شوند

مطمئن شوید MathJax فعال است و هشدارهای renderer را بررسی کنید. برای سندهای بزرگ مقدار timeout را افزایش دهید:

BASH

```
mrs-md2pdf input.md -o output.pdf --timeout-ms 90000
```

فایل HTML میانی را ذخیره کنید:

BASH

```
mrs-md2pdf input.md -o output.pdf --debug-html output.html
```

سپس `output.html` را در مرورگر باز کنید و ساختار و CSS تولیدشده را بررسی کنید.

بررسی Preflight فایل PDF

برای PDFهای مهم و عمومی، بعد از ساخت exampleها یک preflight سریع اجرا کنید. این بررسی چند صفحه نماینده را raster می‌کند، فونت‌های embed شده را فهرست می‌کند و هشدارهای syntax مربوط به PDF را قبل از انتشار ثبت می‌کند:

BASH

```
python scripts/check_pdf_preflight.py \  
  examples/GUIDE.en.pdf \  
  examples/GUIDE.fa.pdf \  
  --pages 1,2,3 \  
  --output build/pdf-preflight.json
```

خروجی تصویری تمیز همچنان معیار اصلی است، اما گزارش preflight محل تکرارپذیری برای گرفتن regressionهای مربوط به فونت، rasterization و parserهای PDF فراهم می‌کند.

پانویس برای ارجاع، یادداشت فنی و توضیح تکمیلی مناسب است.^۲

- ➡ برنامه Mardas MD2PDF به جای رسم مستقیم هر پاراگراف روی canvas فایل PDF، از Chromium برای layout استفاده می‌کند. این انتخاب باعث پشتیبانی بهتر از CSS print، متن ترکیبی، خروجی SVG MathJax، جدول‌ها، تصاویرهای محلی و کدهای هایلایت شده می‌شود.
- بلوک پانویس نزدیک همان ارجاع درج می‌شود و دیگر به انتهای سند منتقل نمی‌شود.
- پانویس چندخطی پشتیبانی می‌شود.
- متن Markdown داخل پانویس حفظ می‌شود.
- توجه کنید که inline code مثل `@page`، `$x$` و `[^id]` وقتی داخل backtick نوشته شود خوانا و literal باقی می‌ماند.

چک لیست نهایی انتشار

قبل از انتشار PDF مهم، این موارد را بررسی کنید:

- ☒ عنوان جلد، زیرعنوان، نویسنده، تاریخ و metadata.
- ☒ زبان و جهت سند.
- ☒ عمق و لیبل‌های فهرست مطالب.
- ☒ صفحه‌های دارای فرمول.
- ☒ صفحه‌های دارای code block و inline code.
- ☒ صفحه‌های دارای تصویر محلی.
- ☒ جدول‌هایی که محتوای پهن دارند.
- ☒ پانویس‌ها و لینک‌ها.
- ☒ شماره‌گذاری footer بعد از جلد.
- ☐ بازبینی بصری نهایی در PDF viewer.
- ☐ هنگام انتشار نسخه tag شده، چک لیست release در `docs/RELEASE.md` کامل شده باشد.